

Eden's DDR4 guide

This guide has been written in response to the seemingly large amount of misinformation regarding DDR4 overclocking, such as made up “meme” timing rules/relations and misconceptions about the importance of certain timings. It also seeks to somewhat address the lack of a centralized source for DDR4 IC overclocking characteristics; the guide lists overclocking characteristics for the most common (and some less common but interesting) ICs.

It is assumed that you are reasonably familiar with navigating your motherboard's BIOS setup already. You will also want to be ready to clear CMOS, in case certain settings cause a no POST situation. The guide has a focus on Intel, though some sections address AMD as well. In general, most parts of memory overclocking are similar between the two, aside from maximizing frequency.

You are not expected to read the entire guide. The Overclocking procedure section is the only part you need to go through to establish your overclock, but using the IC identification guide in combination with the IC characteristics for your DRAM may speed up the overclocking process and give you a better idea of what to expect.

Table of contents

[Introduction](#)

[Stability tests and utilities](#)

[Overclocking procedure](#)

[General overclocking procedure](#)

[Finding maximum frequency](#)

[Intel 6th–10th gen](#)

[Intel 12th and 13th gen](#)

[Ryzen 3000/5000 chiplet](#)

[General advice](#)

[Second timings](#)

[tRRD and tFAW](#)

[tRFC and tREFI](#)

[tRTP](#)

[tWR](#)

[tWTR](#)

[First timings](#)

[tRCD/tRP](#)

[tRAS/tRC](#)

[tCAS](#)

[Third timings](#)

[tRDRD and tWRWR](#)

[tRDWR](#)

[tWRRD](#)

[DDR4 IC characteristics](#)

[SK Hynix](#)

[4 Gbit MFR](#)

[4 Gbit AFR](#)

[8 Gbit MFR](#)

[8 Gbit AFR](#)

[8 Gbit CJR](#)

[8 Gbit JJR](#)

[8 Gbit DJR](#)

[16 Gbit MJR](#)

[16 Gbit AJR/CJR](#)

[Micron](#)

[4 Gbit revision A \(Z80A\)](#)

[8 Gbit revision A \(Z91B\)](#)

[8 Gbit revision B \(Z01A\)](#)

[8 Gbit revision D \(Z01B\)](#)

[8 Gbit revision E \(Z11B\)](#)

[8 Gbit revision H \(Z11C\)](#)

[16 Gbit revision B \(Z22A\)](#)

[16 Gbit revision E \(Z32D\)](#)

[16 Gbit revision F \(Z42B\)](#)

[Samsung](#)

[4 Gbit D-die](#)

[4 Gbit E-die](#)

[8 Gbit B-die](#)

[8 Gbit C-die](#)

[8 Gbit D-die](#)

[8 Gbit E-die](#)

[16 Gbit Samsungs](#)

[Nanya](#)

[8 Gbit B-die](#)

[8 Gbit C-die and F-die](#)

[IC identification guide](#)

[Corsair](#)

[G.Skill](#)

[Kingston](#)

[Crucial](#)

[Bare modules \(including OEM modules\)](#)

[Additional information](#)

[Tips and troubleshooting](#)

[General frequency advice](#)

[Tightening any timing results in instability](#)

[tREFI doesn't run high values](#)

[Advanced tweaks](#)

[Termination resistances and reference voltage](#)

[Intel 6–10th gen tweaks](#)

[Ryzen tweaks](#)

[DDR4 timing summary](#)

[tCCD](#)

[tCAS/CL](#)

[\(t\)CWL](#)

[tRRD/tFAW](#)

[tRCD](#)

[tRP](#)

[tRAS/tRC](#)

[tRTP](#)

[tWR](#)

[tWTR](#)

[tREFI](#)

[tRFC](#)

[Micron revisions and design IDs](#)

[Meme timing rules](#)

[Changelog](#)

Introduction

Double Data Rate (DDR) memory, as suggested by the name, transfers data twice per clock cycle. DDR4-3200 will thus run at a clock speed of 1600 MHz and have a transfer rate of 3200 MT/s (mega transfers per second). Some memory manufacturers incorrectly advertise the transfer rate as the clock speed, meaning you might see DDR4-3200 memory being sold as “3200 MHz” when it is actually running at 1600 MHz. This guide exclusively uses MHz to refer to the actual clock speed, as displayed in utilities like CPU-Z or HWiNFO. Normally the value used in BIOS will be the transfer rate, so simply double any stated values to get the value to use in BIOS.

DDR memory timings are typically measured in clock cycles and not absolute latency. This means that for the same timing latency, you'll need to adjust the timing tick value proportionally when adjusting frequency. For example, 16-16-16 timings at 1600 MHz (1.6 GHz) have the same latency (10 ns) as 20-20-20 timings at 2000 MHz (2.0 GHz). Thus, it is always worth pushing frequency higher as long as the timings increase linearly with the frequency, since this allows for higher bandwidth.

The following formulas can be used to convert between timings in ticks and absolute latency:

$$\text{timing (nCK)} = \text{latency (ns)} \times \text{frequency (GHz)}$$

$$\text{latency (ns)} = \text{timing (nCK)} / \text{frequency (GHz)}$$

This guide largely uses absolute latencies in nanoseconds to make the recommendations applicable to multiple frequencies, so it helps to familiarize yourself with the units and conversion. Consider the following two examples:

A timing of 16 at DDR4-4000 (2 GHz) would have a latency of 8 ns, and a latency of 300 ns at DDR4-3600 (1.8 GHz) would require a timing value of 540 ticks.

When converting absolute latency to timings, it is best to round up if the result is not a round number.

One of the most central aspects to DRAM overclocking is the ICs or memory chips used on the memory modules you are overclocking. Contrary to popular belief, third party module manufacturers like Corsair and G.Skill don't make the DRAM used on their modules themselves, and instead purchase it from first party manufacturers like Samsung, Micron and SK Hynix.

While there are often significant differences in the overclocking properties between DRAM from the different manufacturers, different revisions, “dies” or steppings from one manufacturer can also be vastly different. For information on how to identify memory ICs, see [DDR4 IC Identification](#)

One property that can be different between different DRAM ICs is the density or capacity. This is typically measured in gigabits. For DDR4, typical densities are 4 Gb, 8 Gb, and 16 Gb. For comparison, 1 gigabyte (GB) is equivalent to 8 gigabits (Gb, note the difference), meaning a 4 Gbit IC would have a capacity of 0.5 GB per chip. Multiple chips are then combined into a rank on DDR4 modules, where one rank typically consists of 8 ICs for normal consumer modules. Note that one module can consist of either one or two ranks, meaning that the same capacity ICs can be used for two module sizes depending on the number of ranks.

The following relation generally exists between density, ranks and total module capacity:
 $\text{Capacity (GB)} = \text{ranks} * \text{density (Gbits)}$

For example, a single rank module with 4 Gbit ICs would have a capacity of 4 GB, a dual rank module with 8 Gbit ICs would have a capacity of 16 GB, and a single rank module with 16 Gbit ICs would also have a capacity of 16 GB.

It is further worth noting that ranks are only relevant on a per-channel basis for performance and memory controller load. To get the ranks per channel, add up the ranks for the modules in each channel. 2 single rank modules per channel (4 total on a dual channel platform) and 1 dual rank module per channel would both result in 2RPC (2 ranks per channel). 2RPC is sometimes referred to as “dual rank” and 1RPC as “single rank”, though I prefer to avoid this nomenclature due to potential confusion between ranks per channel and ranks per module. Note additionally that there is no such thing as “dual rank kits” or similar, as the number of ranks per channel for a kit will depend on how it is installed/configured.

The manufacturer, density and revision/stepping/die are generally used to identify ICs and distinguish their overclocking capabilities. For example, Samsung has 8 Gbit B-die and 8 Gbit C-die which have distinct overclocking characteristics, similar to how their 4 Gbit E-die and 8 Gbit E-die also have different overclocking characteristics. Note that the revision names don't inherently indicate overclocking performance; E-die may be better than D-die for one density, but worse for another.

Stability tests and utilities

It is essential to stability test your overclock when using it for daily operation. Not only can an unstable memory overclock disrupt your daily usage (with crashes, bluescreens and other more subtle issues), it can also cause corruption of data, programs or your operating system, which in the worst case may prevent your operating system from loading. Note that this can happen when you are setting up and testing your overclock as well. I would thus recommend either

making a separate Windows installation for stability testing, or performing at least some quick testing in a bootable memory test like MemTest86 or stressapptest on a portable Linux installation before booting into your main OS.

How long to stability test for is a question of much debate, and there is no proper conclusive answer. For any arbitrary amount of time, errors could always have appeared if the test were run for a few minutes longer. With that said, 1 hour of testing provides a good baseline for stability. For higher confidence in the memory stability, longer testing times may be used. A common approach is to test for shorter periods of time while working on the tune, and do a final longer test pass. Note additionally that running multiple tests (one at a time) for individually shorter periods of time may provide higher stability confidence than running a single test for a long period of time. For example, instead of running one test for 12 hours, it could be better to run 3 different ones for 4 hours each.

Another aspect to consider is that timings (and likely frequency) scale with temperature to some extent on practically all ICs. For this reason, there is a chance that your overclock will be unstable in higher ambient temperatures if you've tested it at low temperatures. In a case with poor airflow, the extra heat generated by the graphics card during GPU workloads may also heat the RAM up to higher temperatures than during just memory testing. For some extra margin, you can reduce your case fan speed or unplug some of them entirely during memory testing. Some even go as far as running FurMark at the same time as the memory test, though I find this to be overkill.

The following are some popular memory stability tests:

- HCI MemTest (Windows with paid bootable version)
- MemTest86 (bootable, tests 0-9 recommended)
- TestMem5 (Windows, needs a good config to be effective)
- Karhu RAM Test (Windows, paid)
- Google stressapptest (Linux, can be used on a portable USB drive distro)
- Prime95 blend/large FFTs with high memory usage
- OCCT (Windows)
- y-cruncher component stress test
- LinPack (reasonable all-around test, though generally not the best at anything)

I will briefly go over the efficacy of some of these tests. In both my own testing and that of other people, Karhu has generally been found to be the most reliable and consistent, as well as typically fastest or one of the fastest, in detecting unstable memory overclocks. While it may be tempting to judge stress tests by how quickly they detect rather obvious instability (such as that commonly caused by primary timings), the ability to detect intermittent and non-obvious instability is generally more important, as all reasonable memory tests are able to detect obvious instability. It is in this regard that Karhu especially excels, which is why it is my personal test of choice for final testing.

HCI MemTest—while not especially quick at detecting obvious instability—is reasonably good at detecting intermittent instability given enough runtime, and is therefore a good free substitute to Karhu for final testing. The only annoyance is that the free version requires multiple instances to be opened manually, where one instance should be run per thread.

MemTest86 runs outside of the operating system, and therefore makes a great initial test since you don't risk corrupting your main operating system with unstable memory overlocks while stability testing in-OS. Similar to HCI, it is not the fastest at detecting obvious instability, but does the job well enough. The free version is limited to a maximum of 4 passes, which makes it less suitable for detecting intermittent instability due to the runtime limitation. I would advise against the premium version as the money is better spent on Karhu. Tests 0–9 are the main relevant tests to run. Test 10 and 11 take a very long time to run and are not relevant for the kind of instability caused by memory overclocking.

TestMem5 with a proper configuration is a popular in-OS test. While it is quick to detect fairly obvious instability, such as that typically caused by primary timings, it may struggle to detect intermittent instability, such as from tWR. Some popular configurations to use include PCB Destroyer, anta777 extreme and absolut, and the various 1usmus tests. Disregard any mention of these configs being able to distinguish the exact cause of errors. Even if this was reliably the case for the creator, the results do not transfer to different systems. It is also worth noting that TestMem5 can be rather buggy (refusing to run or getting stuck) and features some less-than-ideal English translations.

γ-cruncher in component stress test mode is a highly effective tool for testing memory controller stability. For this, the tests N32, N64 and VST have proven to be the best in my testing. C17 is also reasonable and can optionally be added to the previous 3. For memory stability testing (such as timings), the FFT test is the main relevant one. It makes for a good addition to the previously mentioned tests; I would recommend running it for 30-60 minutes during final testing if a higher confidence in stability is desired. LinPack, prime95 and OCCT also feature tests that are good for testing IMC stability, but I have found little reason to use them as γ-cruncher with the proper component stress tests is more effective.

In addition to the stability tests, there are a number of other tools which aid in overclocking by letting you see the current status of your overclock or monitor system sensors.

Other useful utilities:

- CPU-Z, for viewing timings and info about your modules
- HWiNFO, for monitoring system sensors
- Asrock Timing Configurator or similar utility, for viewing timings (Intel)
- ZenTimings (same as above for Ryzen)

Overclocking procedure

General overclocking procedure

The general overclocking procedure entails optimizing a setting until the point where it is obviously unstable, then backing it off until stability is reached. The advantage of this method compared to starting at stable settings and working your way towards instability is that it typically saves a lot of time; if you start at a stable value and work your way towards instability, you may spend a lot of time stability testing parameters that are nowhere close to the limit. In comparison, if a parameter is unstable, the instability will often show up quickly in tests, allowing you to quickly work your way towards stability.

For timings, this normally means stepping the timing down until no POST* (or the minimum configurable value if applicable), and increasing the timing until stability is reached if unstable. One exception to this is tREFI, where higher values are better for performance. A similar concept applies to frequency, just that you instead want to increase frequency until obvious instability and then step it down until stable.

*POST means that the system successfully initializes without entering safe/recovery mode. It is essentially equivalent to seeing the BIOS screen or being able to get into BIOS. POST does not include booting the operating system. Note that you are likely to encounter no POST a fair bit during the overclocking process. If your motherboard does not have good auto recovery, this will require a CMOS clear, so make sure you are familiar with that process before starting.

Note that you'll want to stability test in between timing and frequency changes. Whether to do this after every timing you've optimized or after a few of them is up to you. Testing after optimizing a few timings may save you time, but it also makes it more difficult to figure out which timing is causing instability, if encountered. For novice overclockers, it may therefore be easier to do timings one at a time and stability test after each one.

Finding maximum frequency

To achieve maximum clock frequency, there are a few parameters that you will want to set depending on the platform (meaning CPU architecture/motherboard) that you are using.

Intel 6th–10th gen

The Intel 10th gen memory controller is generally quite strong, and can typically be expected to reach between 2200 and 2400 MHz with full stability for 1RPC setups, depending on the specific memory IC and CPU sample, with 2RPC setups often not clocking that much worse. Most Z490 and newer motherboards are reasonably good, and shouldn't hold the memory overclock back too much, though there are some exceptions. Older Intel generations are

fundamentally the same architecture, just less refined, and clock memory slightly worse on average for each generation you go back. However, many Z390 and older motherboards are poor for memory overclocking, so it is likely that these will hold you back more than the CPU's memory controller.

The two main voltages that help with memory overclocking for these CPUs are SA voltage (also known as V_{CC-SA} or System Agent voltage) and IO voltage (also known as V_{CC-IO}). 1.40 V SA should be a good start. Even higher voltage might help stabilizing 2200+ MHz or 4RPC at lower frequencies though it generally won't be needed for most setups. Up to 1.50 V should be safe (as specified by Intel), though even higher voltages have not proven to be harmful in medium-term tests. The IO voltage is slightly more complicated as it typically has a sweet spot, beyond which more voltage doesn't help or even makes stability worse. This can be anywhere from the high 1.2 V range to the low 1.4 V range depending on the specific CPU. Generally I recommend starting at 1.30 V. Ambitious users may fine-tune it which could help if a certain frequency is right on the edge of stability.

Intel 12th and 13th gen

These CPUs feature different "gear modes". For 99% of users, gear 1 will be optimal and this guide will thus focus on that. Unlocked 12th gen CPUs should be able to run around or a bit above 2000 MHz in gear 1. 13th gen CPUs are generally slightly better. To achieve these frequencies, the system agent voltage (SA voltage or V_{CC-SA}) will need to be increased. The 1.3 V range is typically a good spot to be in as scaling with higher voltages is minor. It is worth noting that locked (non-K) CPUs don't allow adjusting the SA voltage, which limits them to between 1650 and 1800 MHz. Typically 2RPC doesn't impact maximum frequency much compared to 1RPC on these CPUs.

Again, make sure that gear 1 is selected. On some motherboards, this is a separate setting. MSI motherboards will list multiple identical frequencies, where some specify "G1" or "G2" for gear 1 and gear 2 respectively. For gear 2 to make sense, you need to be running very high frequencies which is beyond the scope of this guide.

Ryzen 3000/5000 chiplet

These CPUs feature the option to run the IMC clock at half the memory clock, similar to the gear modes on Intel. This incurs a latency penalty which tends to negatively impact performance more than whatever extra memory speed desyncing the two clocks may bring. As such, make sure the MCLK to UCLK ratio is set to 1:1. Here, MCLK is the memory clock and UCLK is the memory controller clock, which in 1:1 mode will be the same (e.g. DDR4-3800 will run 1900 MHz MCLK and UCLK).

Ryzen CPUs additionally feature another clock, known as FCLK. Unless the memory speed is very low, the best performance is achieved with this clock equal to UCLK. Depending on the

motherboard, this may or may not be the default behavior, so make sure to check that they end up at the same value. The FCLK will typically reach a maximum clock speed somewhere around 1900 MHz which with the synced 1:1 operation limits memory speed to the same clock. Use this frequency as your target. Slightly higher, such as 1933 or 1966 MHz may in rare cases also be fully stable. If 1900 MHz does not POST, it could be worth trying slightly higher frequencies since some CPUs have "FCLK frequency holes". If 1900 MHz or above is not stable, 1866 MHz is usually a safe option, assuming the motherboard or memory isn't a limiting factor.

The main relevant voltages are the two VDDG: IOD and CCD. VDDG IOD of 1.05 V tends to be good, while a good value for CCD would be around 0.90 V. These voltages are derived from the SOC voltage using LDOs, and the SOC voltage thus needs to be slightly higher than the highest of these voltages to achieve the desired values. SOC of 1.10–1.15 V should be sufficient.

One should note that the procedure for stability testing the infinity fabric stability is a bit different compared to standard IMC stability testing. For infinity fabric testing, an IMC test such as y-cruncher N32/N64/VST or prime95 large FFTs should be run at the same time as a workload that loads the PCI-E bus, such as the OCCT GPU VRAM test. When running these, look for WHEA errors as reported in HWiNFO or OCCT. Since these types of errors are not catastrophic, 30 minutes is a reasonable amount of time to test infinity fabric stability for. To emphasize, this test only needs to be run when initially maximizing the frequency and infinity fabric, and not for subsequent timing tuning.

General advice

The limiting factor for frequency will be the weakest factor in the memory subsystem, which could either be the CPU's memory controller, the motherboard, the memory ICs or the memory module PCBs. It may in some cases also be a combination of these, such as if the motherboard BIOS is poorly optimized for certain module PCBs or ICs. This can particularly be the case on Z390 and older.

For stress testing maximum frequency, it is generally a good idea to run a more CPU-focused stress test in addition to the memory test, especially if pushing the frequency closer to the above stated common frequency limits of the various platforms. Good options for this include prime95 large FFTs or blend test, y-cruncher component stress test with tests VST, N32 and N64, or a modern variation of LinPack that uses AVX. Note that you should run the CPU test and memory test one at a time, and not at the same time.

For finding maximum frequency, all non-primary timings should be on auto and command rate should be set to 2T. On AMD, 1T with gear-down mode (GDM) enabled is also fine. I recommend leaving XMP disabled and typing in timings manually. 1.35 V DRAM voltage is a good all-around voltage to use, though some ICs may benefit from higher or lower voltage. If your XMP voltage is higher than 1.35 V, you can also safely use that.

The following primary timings may be used when finding maximum frequency:

DDR4-3600 and lower	18-22-22-44
DDR4-3601–4000	20-24-24-48
DDR4-4001–4400	22-26-26-52
DDR4-4401+	24-28-28-56

Note that the order is tCAS-tRCD-tRP-tRAS. On AMD, set both tRCD timings the same initially. Don't worry if those timings seem high since we'll get to reducing them in later steps.

Second timings

We start by optimizing the second timings because these have the largest impact on performance, and also have the highest chance of making other timings unstable when tightened. It thus makes the process both easier and more impactful to adjust these first.

tRRD and tFAW

Asus name: RAS to RAS delay and FOUR ACT WIN time

Start with 4-4-16 (tRRD_S-tRRD_L-tFAW) and increase the tRRD_L timing if unstable. If tRRD_L as high as 8 is still unstable, try increasing tRRD_S or tFAW and if that helps, attempt lowering tRRD_L again. 95+% of memory modules will be able to run tRRD_S 4 and tFAW 16, so only in rare cases will you need to increase these.

These may in some cases impact the maximum frequency slightly, so it could be worth trying a slightly lower memory frequency if 4-x-16 tRRD is unstable. High tFAW and tRRD can also help mask other instability, such as that caused by a poorly overclocking motherboard. In general, these timings have such a large impact on performance that it can be worth dropping frequency by a fair bit in order to achieve tight tFAW and tRRD_S.

tRFC and tREFI

Asus name: REF cycle time (1) and Refresh interval

These timings control for how long the DRAM refreshes and how often it refreshes respectively. Since the DRAM is unable to complete other operations when refreshing, lower tRFC and higher tREFI is better for performance.

In most cases, the gap between no POST and fully stable is very small for tRFC; I recommend stepping down tRFC in steps of 10 or 20 until no POST, then setting it 20-30 ticks higher than the lowest value that did POST depending on how good your airflow around the memory is. As a starting point you can either use the auto timing value, or a value a bit above the typical tRFC for your IC. If your motherboard has good auto recovery, performing a binary search may also be a viable option.

For tREFI, 65k should be a safe value to use unless the memory is extremely poorly cooled, like in a case with very limited air intake or with largely passive cooling. For these setups, a value of 42069 typically works. For 11th gen and older, 65535 is the limit, while 12th gen and newer allow values up to 262k (though some motherboards are still limited to the old value of 65k).

tREFI is extremely temperature sensitive, and it may thus take many hours before instability shows up since the system needs to heat up fully. If you want to push above 65k, I would recommend leaving this for last. Also note that it is generally not effective to make small adjustments to tREFI. A 10 °C temperature increase will typically require halving the tREFI, so if you encounter errors not long after starting stability testing, you may need to decrease tREFI a lot to account for the memory modules heating up more the longer you run the test.

tRTP

Asus name: READ to PRE

Start at 6 and increase if unstable. On Intel, leaving tRTP on auto and setting it using tRDPRE (if available) may produce more consistent results.

tWR

Asus name: WRITE Recovery

Instability caused by this timing often takes hours to show up, and the performance benefit is only moderate. I recommend either leaving it on auto or tuning it last. If tuning, start at 10. Usually best to increase in steps of 2 as a single step might only improve stability slightly.

In some scenarios on Intel motherboards, it is better to leave tWR on auto and control its effective value through tWRPRE (if available), where $tWRPRE = tWR + CWL + 4$. Since I recommend leaving tWR for last, you will have tuned CWL already when you get to it.

tWTR

Asus name: WRITE to READ

On Intel, you want to leave these timings on auto as they are controlled by the tWRRD timings which will be covered later.

On AMD, finding the minimum POSTable value for each of these and running them 1 tick higher will nearly always be stable. If you prefer to avoid no-POST situations, 4-8 (S-L) is a good start for Samsung ICs. For Hynix, 2-10 is common, and Micron can usually go as low as 2-6. These

timings do very little for performance, so I would not recommend tuning them to the last tick. They may even be left on auto without too much of an impact on performance.

First timings

tRCD/tRP

Asus name: RAS to CAS delay

On Intel 10th gen and older, these timings are linked. On certain motherboards, this may also be the case on later Intel generations. For AMD and Intel 13th gen, tRCD is split into tRCD(RD) and tRCD(WR), where tRCD(RD) takes on the conventional value of tRCD.

Minimum values for these depend largely on the IC. Step down independently until no POST and work your way towards stability. Samsung B-die and Nanya 8 Gbit are the only common ICs where these timings scale at all with voltage. tRCD(WR), if present, will almost always be stable at the minimum value of 8.

tRAS/tRC

Asus name: RAS ACT time

These effectively control the same timing. On AMD, set tRAS to 21 and adjust tRC instead. How low it will go mainly depends on the IC. Micron ICs need high tRC (often ~60 ticks at 1900 MHz), while Hynix and some Samsung ICs can run much lower tRC (~40 ticks). 60 is a good starting point but it is worth trying lower.

Intel simplifies things by omitting tRC and only using tRAS. 28 tRAS usually works on all non-Micron ICs. tRAS on Micron ICs varies depending on the sample and frequency, but typically ends up somewhere around 40 at common frequencies. If your Intel motherboard has a tRC setting, it is fake and should be left on auto.

For Intel 11th gen and newer, lower than 28 tRAS is possible depending on the motherboard, though the performance benefit tends to be rather negligible.

tCAS

tCAS, also known as CL or CAS latency, is typically the main or only timing that scales with memory voltage. However, the impact it has on performance is often very minimal (even though it is the main timing that third party memory sellers like to advertise). I will leave it up to the end user to decide if it is worth pushing high voltage to get this largely insignificant timing 1 or 2 ticks lower. Do however note that some ICs may not like high memory voltage.

How low it'll go varies greatly depending on the specific, IC, sample and voltage, so there are no general recommendations. Step it down until no POST and increase it from there until stability is reached.

The CAS write latency or CWL is a similar story to CL. You may leave it on auto or tune it after tuning CL. Note that if you do want to tune CWL, you should do so before the later tRDWR and tWRRD timings; "leaving it for last" will complicate things.

Third timings

On Intel, I've found that if the tRDRD, tWRWR and tRDWR timings POST, there is a very good chance of them being fully stable. Thus, it could potentially save you a lot of time to only tune them for POST-ability and then stability test groups of them at a time. However, be prepared to have to troubleshoot which one is causing the instability if it does turn out to be unstable.

tRDRD and tWRWR

On Intel, start with 6-4-6-6 (sg-dg-dr-dd), then 7-4-7-7, 8-4-8-8 and so on if unstable. Do this for tRDRD and tWRWR individually, not together. The dr and dd timings only matter if running more than 1 rank per channel. These may sometimes run 1 tick higher or lower than the sg timing, so it could be worth playing around with setting them separately from sg.

Note that neither the RDRD/WRWR nor sg/dr/dg timings are linked in any way; they simply happen to run similar values.

For AMD, the timings ending in SCL serve the same function as the Intel _sg timings. Running these at 2 is optimal but higher is often required. 4 should work with most ICs.

tRDWR

This timing will largely depend on the difference between the CAS (read) latency and CAS write latency. Typically tRDWR will run at around CL - CWL + 9, though it may sometimes need to be significantly higher or lower. For Intel, all 4 tRDWR timings generally run the same value.

tWRRD

For Intel, these are the timing that controls the effective tWTR timings, while AMD has the tWTR timings directly and therefore no tWRRD sg/dg. Step them down from the auto value until no POST, and then set them 1 tick higher than the minimum that POSTed; it should be stable in nearly all cases.

The tWRRD dr and dd timings are independent and can typically run values of 6 or lower.

Note that the reported tWTR values in programs such as Asrock Timing Configurator on Intel motherboards are 2 lower than the actual effective values. Thus, you may see reported timings of 0 or other odd values, especially with Micron ICs.

DDR4 IC characteristics

SK Hynix

A common feature of 4 and 8 Gbit Hynix ICs until 8 Gbit DJR is that they require high tRRD_L. 5 may not benefit stability compared to 4. Requiring 6 or 7 is typical. tWTR_L often needs to be fairly high at 9 or higher, while tWTR_S can go low.

4 Gbit MFR

Very common on early DDR4 kits from third party brands, especially during 2014 and 2015. Less common in 2016.

At higher voltages, frequency is commonly limited to 1600 MHz. With very low voltage (often lower than 1.2 V), some samples may reach as high as 1800 MHz. Will typically run 15-15-15 primary timings in the 1.3 V range at 1600 MHz, with around 170 ns tRFC. tRCD supposedly goes lower than tRP on platforms with independent control.

4 Gbit AFR

Found on many higher speed or tighter timing bins (for the time) kits from 2016. Also not uncommon on generic bins like DDR4-2400 16-16-16. Common on Hynix OEM modules from the time.

Similar to MFR, but with better clock speed characteristics. Max is typically at least 1800 MHz, and may go as high as 2000 MHz and above. Maximum frequency is not impacted significantly by voltage. tCAS scales well with voltage and therefore depends on both bin and voltage, while tRCD/tRP doesn't scale with voltage and mostly depends on the bin. tRCD/tRP should run around 10 ns. Around 170 ns tRFC is typical.

8 Gbit MFR

Found on a lot of DDR4-3000 kits, mostly from 2018 and earlier.

Frequency typically maxes in the 1700 MHz range, though some samples will go into the 1800 MHz range. Voltage for max frequency seems to sweet-spot, so running very high or very low voltage might negatively impact maximum frequency. tRCD/tRP runs quite low for 8 Gbit ICs, often being able to do 10 ns. tCAS varies depending on the voltage and frequency, but will typically run a few ticks lower than tRCD at around 1.4 V. Needs 280-290 ns tRFC.

Loosening tCCD (that is the tRDRD and tWRWR sg/dg (Intel) or SC/SCL (AMD) timings may help achieve higher clocks. However, tCCD_S (tRDRD/tWRWR SC/dg) has a large impact on bandwidth, so it may or may not benefit performance. Could likely be worthwhile on AMD to hit 1900 MHz synced, but performance testing and comparison is recommended.

8 Gbit AFR

Used in various DDR4-3000 and lower bins from 2018 and earlier. Also common on Hynix OEM modules from 2016 to mid 2018.

On a capable platform, 1800 MHz* should be attainable. Frequency scales slightly with voltage, so better samples may be able to hit 1900 or even 2000 MHz at 1.4-1.5+ V. Needs around 11 ns tRCD/tRP. tCAS should run around 9 ns or slightly tighter at 1.4 V. Only tCAS scales significantly with voltage and tRCD/tRP may even scale very slightly negatively with high voltages.

tRFC should run 250-260 ns. Tightening tRFC may sometimes require one tick looser tRRD_L and/or tRCD/tRP if they are close to the limit. This is likely worthwhile for performance, but test yourself on your setup.

*It has been theorized that the DDR4-2133 JEDEC bin, which is typically what is used on third party modules, clocks significantly worse. OEM modules of the DDR4-2400 JEDEC bin have all clocked to 1800+ MHz. It is also likely that Z390 and older motherboards have poor compatibility with AFR, and since these were what was mainly used for testing when AFR was common, this could explain poor results.

The tCCD phenomenon observed on MFR may also apply to AFR, though it is very unlikely to be worthwhile with AFR due to it clocking higher at tight/auto tCCD.

8 Gbit CJR

Used on many of G.skills DDR4-3200+ kits with typical timings from 2018 onwards. Also somewhat common among other manufacturers. Present on 2018+ Hynix OEM modules.

Similar to AFR above, but typically does 10-20 ns tighter tRFC and clocks higher. Maximum frequency tends to be between 1900 and 2200 MHz. tRCD will often run 1 or sometimes 2 ticks lower than tRP.

High voltages may be damaging to the IC. Some users report being fine at 1.45 V for 2+ years, while others report damage from voltages as low as 1.42 V. It's unclear whether or not it's actually the voltage that is damaging the ICs, given the large discrepancy between reports. However, unlike Samsung C-die, I am not aware of CJR dying at 1.20 V, which suggests it may actually be the voltage causing the issues. For users wanting to play it safe, it is likely best to stick to 1.40 V max.

8 Gbit JJR

Not very common. Similar to CJR above but with tRP instead normally being able to go lower than tRCD, and tRFC being slightly looser.

8 Gbit DJR

Used on nearly all high clock 8 GB modules with loose XMP timings from the late DDR4 era. Also common on more standard bins like DDR4-3600 18-22-22 or similar, especially from Kingston.

Quite different from previous Hynix ICs. Known for clocking very high; the motherboard, CPU or PCB should be the limit long before the ICs themselves. Runs 4-4-16 tRRD/FAW even at high frequencies. tRCD/tRP is a bit loose, typically needing 11 ns or higher. tCAS scales well to 1.6 V and above, where it can often reach 8 ns. 1.6 V can be presumed to be safe, given its usage on XMPs for DJR kits. tRFC is quite low for 8 Gbit ICs, in the low 200 ns range.

16 Gbit MJR

Found on some earlier 16 Gbit-based DDR4-4000 18-22-22 kits, mainly from G.skill. Also used for lower spec XMPs. The DDR4-4000 XMP is about the best you can expect in terms of timings. Frequency might go a little bit higher. Many samples will require 18-22-22 timings at even lower frequencies than 2 GHz. Also needs very high tRFC, probably the highest of all DDR4 ICs, at 380 ns. Some samples may go a bit lower.

16 Gbit AJR/CJR

Used on many 16 and 32 GB modules rated for high speeds with loose timings from the late DDR4 era. Similar to DJR above but with looser tRFC. Again, 1.55-1.60 V can be presumed to be safe given its usage on XMPs for kits based on these ICs.

Micron

Micron ICs should generally be able to run 4-4-16 tRRD/FAW at typical frequencies. Some may need tRRD_L 6 at high frequencies. Many also run low real or effective tWTR timings such as 2-6.

4 Gbit revision A (Z80A)

Mostly found on very old 4 and 8 GB modules, mainly from Crucial. Can appear on modules produced as late as 2016. Nanya 4 Gbit B-die should be equivalent.

Frequency typically maxes somewhere around 1600 MHz, with 1500–1700 MHz being a common range. Primary timings are similar to 8 Gbit Samsung B-die, often running 13-13-13 at 1600 MHz around 1.4 V. Only tCAS scales significantly with voltage. tRFC typical for 4 Gbit ICs, running 160–170 ns.

An ideal choice for Intel chipsets with locked memory frequency or X99 due to the low timings, but subpar for modern unlocked platforms that can clock higher.

8 Gbit revision A (Z91B)

Barely clocks beyond the typical JEDEC bins. Timings are poor as well. Avoid at all costs.

8 Gbit revision B (Z01A)

Reasonably common on older Corsair and Crucial modules rated for DDR4-2400 and similar speeds. D9TBH FBGA code for standard ICs.

Frequency scales inversely with voltage at a significant rate. Pushing voltages beyond 1.4 V will likely limit you to the 1700 MHz range or lower. Timings are unexceptional as well. Expect around 19-21-21 at 1700–1800 MHz with 260 ns tRFC. Only tCAS scales significantly with voltage, meaning you are better off pushing the frequency to the limit at low voltage, even if it requires loose tCAS.

8 Gbit revision D (Z01B)

Likely used on modules as late as 2022. Similar to B-die above, but better for both timings and frequency.

8 Gbit revision E (Z11B)

Used on nearly all higher speed (1500 MHz+) 8 Gbit-based Crucial modules from 2019 onwards. Also not uncommon on Corsair modules. Used on Micron OEM modules mainly from 2019 and 2020.

On compatible platforms, known to clock to 2500 MHz and higher for reasonable samples. Even OEM modules can reach high clock speeds, though not always as high as good consumer brand modules. May require tweaking of the DRAM VTT voltage if going above 1.35-1.45 V, where the VTT voltage often runs best around 0.68-0.74 V. Many Z390 and older motherboards strongly dislike it and will not clock high at all.

tCAS is generally very consistent across modules, with OEM modules often being very close to or identical to medium bin consumer brand modules. Even fairly poor modules should run 8 ns CL at around 1.5 V. Typically scales at approximately 80 mV per tick of tCAS, up to the higher 1.5 V range. May continue to scale slightly worse to far higher voltages. 1.55-1.60 V has not proven to be damaging to the IC so far given reasonable cooling.

tRCD depends highly on the bin, where better Crucial bins can run well below 10 ns, while OEM modules may need 12 ns. Crucial's DDR4-3000 and 3200 bins, as well as other consumer brand modules, typically run around 10 ns, in part depending on temperature.

tRP can often be many ticks lower than tRCD. Effective tRC varies between 25 and 33 ns largely depending on bin. Minimum tRFC is usually around 300 ns or a bit lower. Only tCAS scales significantly with voltage.

8 Gbit revision H (Z11C)

Common in DDR4-3000 XMP kits from 2017. Used on Micron OEM modules as well. Not seen much on modules outside of 2017.

Can clock to 1750 MHz and beyond, given a suitable platform. May not even hit 1600 MHz on many Z390 and older. At 1750 MHz, CL 15 will typically require around or a bit more than 1.4 V, while tRCD should run 19–20. tRP may run multiple ticks lower than tRCD. Only tCAS scales significantly with voltage, though see below.

Tightening tRFC can significantly increase the voltage requirement at a specific CL, though given the low significance of the CAS latency on modern platforms, it is usually worth trading CL for tRFC. Some samples can run 190 ns and a bit lower, which is very low for 8 Gbit ICs.

16 Gbit revision B (Z22A)

Used on various third party kits from the later DDR4 era, ranging from fairly basic DDR4-3600 18-22-22 to some insane bins from Crucial like 4400 19-19-19 and even 5100 19-26-26. Not especially common on OEM modules, but does exist with the corresponding part number.

Generally considered to be the best 16 Gbit IC, as it is able to run high clock speeds, and depending on the bin, low timings. At the bottom end, there are modules that don't tighten primary timings beyond the 3600 18-22-22 1.35 V XMP (12 ns tRCD) without increasing the voltage. On the other hand, really good samples can run 14-15-15 (<9 ns tRCD) at the same speed in the 1.4 V range.

The IC is likely to max out the IMC before becoming the limit itself, easily reaching above 2500 MHz on capable IMCs and motherboards. Only tCAS scales notably with voltage, with good scaling up to at least the 1.5 V range. Scaling higher than that depends on the sample and cooling. On some samples, tRP goes multiple ticks lower than tRCD, while on others, they are close to the same. Expect 300 ns tRFC or a bit lower.

16 Gbit revision E (Z32D)

Found on third party kits from around 2022. At best, it makes its way into 3600 18-22-22 XMPs, but it also commonly appears in lower speed ones.

Frequency maxes out around 2000 MHz, with some samples not going much above 1800 MHz. Timings mostly seem similar to lower tier Z22A (i.e some samples not tightening much beyond the 3600 18-22-22 XMP at 1.35 V), though it's possible that some modules are a fair bit better than that, as seems to be the case for many Micron ICs. However, with it being an inferior IC compared to Z22A and not being binned much by neither first nor third parties, one would be unlikely to find any significantly better samples. tRFC at best does 310 ns.

16 Gbit revision F (Z42B)

Seems to most reliably be found in the Crucial Pro DDR4-3200 JEDEC kits from 2023.

Frequency is even worse than Z32D; at best, you might be able to hit 1800 MHz. Timings vary a bit, from modules that don't tighten beyond the upper 11 ns range for tRCD and 10 ns for tCAS at 1.35 V, to ones that go at least one nanosecond tighter.

Samsung

All somewhat recent Samsung ICs should run 4-4-16 tRRD. 4-8 tWTR should also be fine most of the time at 2000 MHz and lower.

4 Gbit D-die

Mainly used on modules from mid 2015 and earlier.

Most modules seem to max out in the 1700 MHz range, though some early high speed (for the time) DDR4 bins of this IC exist, with XMPs of 1800 MHz and above. tRCD/tRP should run 10 ns or slightly higher. tCAS can be 1–3 ticks lower than tRCD/tRP depending on the sample and voltage. This is one of the few odd ICs that require 5 tRRD_S. 5-5-20 tRRD/tFAW should work. Expect around 180 ns tRFC.

4 Gbit E-die

Used on most of the top kits from late 2015 and 2016. Also appears on some lower bins. Used on Samsung OEM modules. M378A1G43EB1-CPB is strongly recommended as cost-effective 8 GB DDR4 modules, often clocking similar to 8 Gbit Micron E-die, while also offering the benefits of 2 ranks.

The IC itself should clock to 2500 MHz and beyond, but other factors may limit the clock. To push much above 2000 MHz, it is essential to get modules on A2/B1 PCBs. tCAS scales very well to extreme voltages, with other timings not scaling significantly or at all. tCAS and tRCD/tRP depend largely on bin, with significantly lower than 10 ns tRCD/tRP being uncommon. tRFC is quite tight at 130–160 ns.

There are no signs of even 2.0 V causing damage to the IC if properly cooled, though such high voltages may not be good long-term for the CPU.

8 Gbit B-die

Only IC used on kits with an XMP tRCD latency of less than 10 ns for 2017 kits and later, with the exception of Crucial. Also frequently found in kits with standard XMPs from 2017 and earlier, and less frequently after 2017.

The variation among these is insane. At the bottom end, there are samples that top out at 1600 MHz with 16-18-18 timings and don't scale beyond the 1.3 V range, with only tCAS scaling at lower voltages. tRFC is also often bad on these samples, in the upper half of the 200 ns range.

For better B-die, which should be all samples with XMPs such as 3200 14-14-14, 3600, 16-16-16, 4000 19-19-19 or better, the characteristics are far superior. All primary timings scale

well with voltage. tCAS commonly scales to between 1.55 V and 1.70 V, while tRCD usually doesn't scale above 1.35-1.60 V. In fact, tRCD may start scaling negatively on some samples above 1.5-1.6 V. tRFC scales extremely well with voltage in addition to running very low to begin with. Even with moderate voltages, 150 ns tRFC is attainable, while 110 ns for full daily stability may be possible at voltages above 1.6 V.

These better samples will generally clock very high, often limited by the IMC, assuming a capable module PCB and motherboard. It is common to see 2500 MHz and above in gear 2 on 11th gen Intel Core processors.

Timings vary depending on the module. Most modules of typical "B-die bins" (such as the XMPs listed above) should be able to run 15-15-15 timings at 2000 MHz or 14-15-15 timings at 1900 MHz. 2000 MHz 14-15-15 and 1900 MHz 14-14-14 is common, though a bit more rare. Strong samples should run 2000 MHz 14-14-14, while really strong samples can run 2400 MHz 16-16-16 or 2250 MHz 15-15-15.

In general, tCAS is roughly as strong as tRCD at lower voltages (<1.5 V). However, due to tRCD scaling typically ending at lower voltages than tCAS scaling, it is common for modules to require 1 tick higher tRCD than tCAS at above 1.5 V. At 2000 MHz, a typical sample may see the following timing scaling as voltage is increased:

17-17-17, 17-16-16, 16-16-16, 15-15-15, 14-15-15.

On samples where tRCD scales well at 1.5 V and higher, you may see tRCD running the same as or even a tick lower than tCAS throughout the entire voltage range. For platforms with independent tRCD and tRP control, tRP can often run one or a few ticks lower than tRCD.

In addition to running low primary timings and tRFC, B-die is also known to run other timings at low values, like tRTP, tWR and especially tRDRD_sg. tRTP 6 should be stable at most typical frequencies. tWR 10 often works at lower frequencies, but 12 or 14 may be required for higher frequencies. tRDRD_sg is especially impressive, running 5-6 depending on the frequency and voltage.

There are no signs of even 2.0 V causing damage to the IC if properly cooled, though such high voltages may not be good long-term for the CPU. However, for daily operation, above 1.5-1.65 V often doesn't improve stability.

8 Gbit C-die

Used on most of Corsair's DDR4-3200 16-18-18 and DDR4-3600 18-22-22 kits from 2019 to 2021. Similar for other brands, though generally not as widespread as with Corsair. Used on Samsung OEM modules from late 2016 to 2019.

Frequency typically maxes out between 1900 and 2050 MHz, though some samples will clock as high as 2350 MHz and possibly higher. Primary timings are very poor. Expect around 10 ns tCAS at 1.35 V and 11 ns tRCD/tRP. tRFC is also bad at 310-320 ns. Only tCAS scales with

voltage. tRCD scales negatively with voltage above ~1.4 V, meaning it is usually not worth running higher voltages than this.

There are claims of C-die dying from more than 1.35 V or even just 1.35 V; however, given C-die's tendency to randomly die even at 1.20 V, I would not interpret this as the higher voltage being the cause. Two users have reported running C-die at 1.40 V for long periods of time with no issues, while there are also reports of C-die dying at 1.35 V XMP. I've personally had two sticks die at just 1.20 V, so I think it is safe to assume that voltage is not what is causing some C-die to fail.

8 Gbit D-die

Has largely replaced C-die on third party modules from 2022 and OEM modules from 2020 onwards.

Similar to C-die above in timings, though tRCD doesn't roll over with voltage, meaning that running high voltages is actually practical. Frequency should go much higher, to 2400 MHz and above if the platform permits. tRFC also runs a bit tighter at 290-300 ns.

8 Gbit E-die

Started appearing on standard DDR4 XMPs in late 2022, such as 3600 18-22-22, 3200 16-20-20 and worse.

Don't expect much beyond the 3600 18-22-22 XMP, neither in frequency nor timings. Some samples aren't even able to do that XMP, though these samples shouldn't come in the 3600C18 and 3200C16 rated kits. Only tCAS scales significantly with voltage, and does so to high voltages as well.

16 Gbit Samsungs

None are especially good. A and B-die at least clock a bit, with B-die reaching 2 GHz and above fairly consistently. M-die scales negatively with voltage for frequency even below 1.35 V. Optimal voltage is usually in the 1.2 V range. Don't expect much tighter tRCD/tRP than 12 ns for any of them. tCAS depends on the voltage and sample but is generally among the worst for DDR4 ICs. Fairly high tRFC of 320–330 ns is required as well.

Nanya

8 Gbit B-die

Occasionally occurs on standard timing Corsair bins. Version number 8.31.

Very impressive IC, often performing similarly to typical bins of 2017 Samsung B-die or even better for timings. Clocks fairly high, with some samples doing 2333+ MHz, though others seem to be unstable above 2100 MHz. tCAS, tRCD, tRP and tRFC all scale with voltage similar to on Samsung B-die. tRFC can run around 150 ns, largely depending on voltage. tCAS, tRCD and tRP at a certain voltage seem to vary a fair bit between samples. tRCD/tRP typically needs to be around 2 ticks higher than tCAS. Good samples should run 1900 MHz 14-16-16 in the 1.5 V range, but most will need slightly higher timings. Do however note that Nanya DDR4 gets very warm and may not see much scaling above even 1.4 V depending on the cooling.

8 Gbit C-die and F-die

These are similar to Nanya B-die, but generally slightly worse in all regards. At 1900 MHz, 16-18-19 timings in the 1.4 V range would be typical.

IC identification guide

The manner in which to go about identifying the ICs used on your module largely depends on the brand of that module. For brands not covered, the SPD tab in CPU-Z will often read out the manufacturer of the ICs in the DRAM manufacturer field, though sometimes the module brand has changed this field to their name.

Corsair

Corsair has a version number printed on the sticker on the physical modules, which corresponds to specific ICs. See the following table.

Version	IC(s)
3.2x	Micron 4 Gbit ICs
3.30	Micron 8 Gbit revision A, B and others
3.31	Micron 8 Gbit revision B, E and others
3.32	Micron 8 Gbit revision H, J and R

3.34	Micron 8 Gbit revision E
3.40	Micron 16 Gbit revision B
3.41	Micron 16 Gbit revision B and F
3.43	Micron 16 Gbit revision E
3.44	Micron 16 Gbit revision E (and apparently rarely B)
4.23	Samsung 4 Gbit D-die
4.24	Samsung 4 Gbit E-die ("E-die")
4.31	Samsung 8 Gbit B-die ("B-die")
4.32	Samsung 8 Gbit C-die ("C-die")
4.33	Samsung 8 Gbit D-die
4.34	Samsung 8 Gbit E-die
4.49	Samsung 16 Gbit M-die
4.40	Samsung 16 Gbit A-die
4.41	Samsung 16 Gbit B-die
5.29	Hynix 4 Gbit MFR
5.20	Hynix 4 Gbit AFR
5.39	Hynix 8 Gbit MFR
5.30	Hynix 8 Gbit AFR
5.32	Hynix 8 Gbit CJR ("CJR")
5.38	Hynix 8 Gbit JJR ("JJR")
5.33	Hynix 8 Gbit DJR ("DJR")
5.49	Hynix 16 Gbit MJR ("MJR")
5.40	Hynix 16 Gbit AJR ("AJR")
5.42	Hynix 16 Gbit CJR
8.2x	Nanya 4 Gbit
8.3x	Nanya 8 Gbit

G.Skill

For G.Skill modules made in 2018 and later, there will be a code on the physical sticker of the modules starting with 04. The relevant part here is the last 5 characters, for example 8810B. The 5th last character, in this case 8, is the IC density, where 4 is 4 Gbit, 8 is 8 Gbit and S is 16 Gbit. The 3rd last character is the IC manufacturer: 1 for Samsung, 2 for Hynix and 3 for Micron. The last character is the die revision. 8810B would thus be 8 Gbit Samsung revision B, or "B-die".

Note that the die revision for Micron may at times be weird, though G.Skill generally prefers Samsung and Hynix ICs.

Modules made in 2017 and older without the 04 code likely use the same serial number format as DDR3. In this case, check the serial number, which should start with the date code in yyww format. The 6th digit in the serial number corresponds to the IC manufacturer, with 3 for Micron, 4 for Hynix and 5 for Samsung.

Kingston

Kingston modules come with an 11 digit code on the module sticker, which starts with 4 letters. The 4th letter corresponds to the IC manufacturer: S for Samsung, H for Hynix, M for Micron and N for Nanya. Modules marked K use ICs that Kingston have rebranded (that is, changed the markings on) and may be of any manufacturer, though most commonly Micron.

The next two digits after the first 4 letters indicate the number of ICs on the module: 08 for 8 ICs (normally single rank/sided) and 16 for 16 ICs (normally dual rank/double sided). This, in combination with the module size can be used to figure out the IC density as per the explanation in the introduction.

The 8th and 9th digits are the manufacturing year (21 for 2021, for example). This can be used to make guesses about which specific IC revision the modules used, based on time frames when certain ICs were most common.

Crucial

For Crucial modules, check the model number printed on the physical modules. We are interested in the last part after the dot, for example .M16FE. The first character is the IC manufacturer, which will almost always be M for Micron, since Crucial is a Micron brand. The number indicates the number of ICs on the modules, where 8 ICs are normally used per rank. This, in combination with the module size can be used to figure out the IC density as per the explanation in the introduction.

The letter after the F (though sometimes the F is missing) is the IC revision. .M16FE as in the example would on a 16 GB module thus be 8 Gbit Micron revision E, in a dual rank configuration.

Bare modules (including OEM modules)

Bare modules are modules without heat spreaders, where you can see the ICs themselves. Use the information printed on the ICs to identify their revision. This information can also be used to identify the density, though I find it easier to simply count the number of ICs and determine the density based on this and the module size, as explained in the introduction.

Samsung

Samsung ICs can be recognized by the text SEC and a small QR code. The letter after 5W is the IC revision. For example, ~5WB would be revision B (some kind of B-die).

Hynix

SK Hynix ICs will have SK Hynix written on them. The 3 last characters of the code starting with H5 correspond to the IC revision. For example, ~AFR would be some density of AFR.

Micron

Micron ICs will have the Micron logo printed on them, as well as a 7 character code usually starting with D9. Type this code into the Micron FBGA decoder, available at

<https://www.micron.com/support/tools-and-utilities/fbga>

This will give you the corresponding full part number of the IC, where the last letter after the colon is the IC revision. For example, D9VPP corresponds to MT40A1G8SA-075:E, which is revision E.

Micron ICs may also be sold under the SpecTek brand, in which case they carry the SpecTek logo instead of the Micron logo. The SpecTek logo consists of an S inside of a rhombus.

Relating SpecTek-branded ICs to their corresponding Micron counterpart is a somewhat complex endeavor which I might cover later. For now, see the SpecTek decoder:

https://www.spectek.com/menus/mark_code.aspx

Others

ICs branded with the name of module brand, such as Kingston or ADATA, are rebranded ICs made by one of the major IC manufacturers and not by the module brand. There is no consistent way to universally identify these.

In some cases, the SPD tab in CPU-Z will still read out the real IC manufacturer, in the DRAM manufacturer field. Other times, the module brand will have filled this field in with their name as well.

Additional information

Tips and troubleshooting

General frequency advice

On Intel, it is worth trying both the 100 MHz/1:1 base DRAM frequency/ratio and the 133 MHz/1:1.33 frequency/ratio. Sometimes one will have better stability than the other, even at the same frequency. In rare cases, switching ratio can even result in a higher frequency strap being stable than a previously unstable one with the other ratio.

On Asrock motherboards, enabling XMP can supposedly help with overclocking, even if overwriting all the timings. If enabling XMP when overclocking, I would recommend setting at least the tRFC timing manually in addition to the primary timings, as XMP tRFC can in some cases be too tight when increasing frequency. For 8 Gbit and most 16 Gbit ICs, 350 ns is a good starting point. For 4 Gbits, use 260 ns instead. Note the nanoseconds unit; you will need to convert it to the actual tick value based on the frequency using the formula in the introduction.

Tightening any timing results in instability

Sometimes, you get to a point where tightening any timing results in instability, even for safe/loose timing values that should be stable. This is usually caused by the tighter timings or higher performance/bandwidth destabilizing something else than those timings. As discussed earlier, this can often happen with the tRRD/tFAW timings, since these timings have the biggest impact on bandwidth and performance. It is also not uncommon for tRFC and tREFI. Generally every timing has the potential to cause instability this way, but it tends to be more common the greater of an impact the specific timing has on performance.

Usually dropping the frequency very slightly will alleviate these issues. Another option could be trying things that help stabilize IMC instability, especially when close to the typical frequency limit for the IMC, rather than the RAM. This would include things such as higher IMC voltages, termination resistance tuning and trying a different base ration (1:1 or 1:1.33). In some cases it could also be a previously tuned timing that is causing the issues. Whether or not it is worth loosening this timing would depend. For example, loosening tRRD_L by one tick in exchange for significantly tighter tRFC would likely be worth it, but probably not for tRRD_S or tFAW.

tREFI doesn't run high values

Some samples simply don't like high tREFI, regardless of the cooling. While normally 65k tREFI should be easily attainable if the modules aren't too warm, some modules may be unstable at even 30k tREFI no matter if warm or at room temperature. It is likely that there is some kind of defect in these modules, though since it is not significant enough to cause them to not meet the

spec, they cannot really be considered faulty. In general, it's not that big of a deal as you can still get large performance gains by tuning other timings. Consider that AMD doesn't even allow you to change the tREFI timing.

Advanced tweaks

Termination resistances and reference voltage

Without going into too much detail, adding resistance at the end of a transmission line, known as electrical termination, minimizes unwanted effects that can distort the signal and make it less intelligible. The ideal termination resistance depends on the properties of the signal conductor. In practice, this means that it will vary between motherboards and CPUs, even with one of the two being the same. There is no magic formula or precise predetermined values that will always be optimal; if there were, they would already be used and there would be little benefit to tuning them manually. DDR4 SDRAM devices feature user-configurable ODT (on-die termination), which allows optimizing the termination resistance for the application.

Tuning termination resistances seem to help most on harder-to-run memory configurations, mainly more than 1 rank per channel. On Intel, the benefits seem most noticeable on Skylake and its derivatives (6th–10th gen).

While there are no general best values applicable to all configurations, some patterns can still be observed and used as starting points for further tuning. RTT-WR seems to usually run best at 80 Ω , occasionally preferring 120 Ω . On Intel Z490 (and possibly older), RTT-PARK seems to benefit from lower values, usually between 34 and 60 Ω , though some users have reported improvements with even higher values. Conversely, AMD Ryzen seems to benefit from much higher RTT-PARK resistances. RTT-NOM usually runs best at low values, or even disabled/0.

Some motherboards instead use RZQ-notation, where $RZQ = 240 \Omega$. $RZQ/3$ would thus be 80 Ω , as an example.

AMD Ryzen CPUs additionally feature user-configurable processor on-die termination (ProcODT). This once again varies between motherboards, CPUs and memory configurations. You could try recommendations made by other users with similar configurations, or use the auto value as a starting point and tune it from there.

On Comet Lake (Intel 10th gen), optimized RTTs can yield a 100 MHz frequency improvement or potentially more. Even setting some quick rough values based on the recommendations above will often realize substantial benefits.

In addition to termination resistances, tweaking the reference voltage, often VREF (likely referring to $V_{REF(CA)}$) can often yield similar but smaller gains. The standard value is $0.5 \times VDD$

(DRAM/DIMM voltage) and typically good values to try would be within the 0.495–0.530x VDD range.

Intel 6–10th gen tweaks

On Intel 6th to 10th gen, the RTLs and IOLs can be tuned by stepping them down in pairs. These timings are tuned per rank and per channel. Thus, if you are running 1 RPC in dual channel, you will have two pairs of RTLs/IOLs, one per channel, while with 2 RPC, you will have 4 pairs, and so on.

Note the auto RTL and IOL values for the rank and channel you are tuning, and decrease both by 1 at a time until no POST. Typically values that POST will be stable, so there is no need to stability test after every change. Repeat the process for the other ranks and/or channels.

The IOL value established for the first rank can often be used for subsequent ranks, which may speed up tuning. However, the RTL value always needs to be stepped down by as much as the IOL value, which means that RTLs may not run the same after tuning if they were not the same before tuning.

The previously mentioned ODT and VREF tweaks can help maximum frequency on these CPUs a lot. Here, a method for fine-tuning these values is introduced. With standard/auto RTT ODTs and VREF, increase frequency to the maximum stable, then reduce V_{CC-IO} to a voltage where it is somewhat unstable. You should notice instability within the first minute of your test of choice, while also not being so unstable that it errors right as the test starts, since it can be hard to tell if changes are actually beneficial when it errors right away.

When this somewhat unstable IO voltage has been found, adjust the RTT ODTs one at a time and observe if the change improves stability. I would start out with setting RTT-WR 80, which is often the default value and may therefore not affect stability. In rare cases, 120 can be better, so it could be worth trying. Next, move to RTT-PARK. While testing RTT-PARK, set RTT-NOM to a low value like 34 or 0. Start at PARK 48 and increase or decrease depending on which improves stability. Repeat until both higher and lower values result in greater instability, or no meaningful change. Repeat this process with RTT-NOM, but this time starting at your previous value (34 or 0). If the changes to the RTTs throughout the process improve stability to the point where tests are no longer erroring or crashing within the first minute, decrease IO voltage as needed until that level of instability is reached again.

The same methodology with running a somewhat unstable IO voltage can also be used to find the optimal VREF voltage. This will often be a bit above 0.500x VDD.

In addition to the previously mentioned ODT and VREF tweaking, adjusting rising and falling slopes, as well as their offsets, may offer minor benefits to maximum frequency on these CPUs, especially with multiple ranks per channel. This is largely a process of trial and error which may

take multiple days, and the benefit is at best very minor—around 1 strap higher memory frequency.

This is largely a brute-force process, once again utilizing the same V_{CC-IO} methodology. Start at a value roughly in between the maximum and minimum and increase or decrease it until the best stability is reached. Going in steps of 2 or even 4 initially might help speed the process up. If you find that a range of values all offer equal stability, I would suggest using a value in the middle of that range.

The biggest improvements are generally from the rising slopes, and I would thus recommend starting with these first. After each slope, you can tune the slope offset, which only has values 0 and 1, thus making it significantly easier to tune. When all rising slopes and offsets are done, move on to falling slopes. In my experience, the DATA slope had the biggest impact on 2 RPC, while it had very little impact on 1 RPC.

Finally, I will also mention that some users prefer to use the SA voltage instead of IO voltage to fine-tune ODTs and slopes. However, I find that the IO voltage is more sensitive and thus produces better data.

Ryzen tweaks

On the standard 3000 and 5000 series Ryzen CPUs, you are most likely to be limited by the FCLK frequency (~1900 MHz) when overclocking, which means there isn't much tweaking that can help with achieving higher frequency. One thing that can be done instead is running 1T command rate with gear-down mode disabled. Depending on the motherboard and memory configuration, this will likely not be stable by default, though there are a number of tweaks which may help in stabilizing it.

The relevant parameters to adjust here are the drive strengths and setup times. If power down (PPD) is disabled, the CKE parameters should not matter. In other words, disabling power down should make the process easier since there are less settings to tune. If you do want power down enabled, it could be worth tuning the non-CKE parameters first with it disabled, then enabling it and tuning the CKE-parameters if necessary.

The CS parameters should only matter for more than 2 rank per channel, since CS is the signal that controls which rank is "selected"; therefore, the CS settings are only relevant on multi-rank-per-channel setups. Single rank per channel setups are therefore fairly simple to tune, since the only relevant parameters (assuming power-down is disabled) are the CLK and ADDR CMD ones.

As with termination resistances mentioned above, there are no optimal settings for these – neither drive strengths nor setup times. Start at the auto values and adjust them until higher or lower values result in worse stability. While it may be a tedious process, with enough time, you will hopefully be able to stabilize 1T gear-down mode off.

DDR4 timing summary

Note that this information is not necessary to overclock the memory, and generally won't help you achieve better overclocks. It is simply provided for those curious about the actual function of the various timings.

First, some basic theory. DDR4 is organized in the following hierarchy:
(Channel) - Rank - Bank group - Bank - Row - Column

For reading and writing operations, the row that is to be read from or written to must be open. This is accomplished by sending an Activate command specifying which row in which bank is to be opened/activated. Next, a read or write command can be issued, specifying the bank and column that is to be written to or read from.

Note that different rows can be open at the same time in different banks and ranks; however, only one row can be open at a time in any specific bank. To operate on another row in the same bank, the current open one must first be closed by sending a Precharge command to the bank in question. After the row has been closed by the Precharge command, another one can be opened by another Activate command.

DRAM needs to be periodically refreshed to retain the data stored in it. This is accomplished by issuing a Refresh command. Before a Refresh command can be issued, all banks must be precharged and idle.

(Note: some of the following info assumes AL = 0)

tCCD

tCCD specifies the minimum time between two consecutive column accesses (reads or writes). The tCCD_S (short) timing is for column accesses to a different bank group (Intel: dg, AMD: SC), while the tCCD_L (long) timing is for accesses to the same bank group (Intel: sg, AMD: SCL). It is often split into separate timings for reads and writes, such as tRDRD (Read to Read) and tWRWR (Write to Write). See the datasheet of your CPU for more details.

tCAS/CL

tCAS, also known as CL (CAS Latency) or tCL, specifies the time between when a Read command is issued, and when the first data can be "extracted" from the DRAM.

(t)CWL

CAS Write Latency, known as (t)CWL, (t)WCL or less correctly (t)WL, specifies the time between when a Write command is issued and when the first data can be "sent to" the DRAM.

tRRD/tFAW

tRRD specifies the minimum time between two consecutive ACT commands to different banks. As for with tCCD, S is for consecutive ACTs to different bank groups, while L is for ACTs to different banks in the same bank group. tFAW specifies the duration of a window within which no more than 4 ACT commands may be issued. As this is generally not a limitation in practice, tFAW should be set to $4 * tRRD$ for best performance.

tRCD

tRCD specifies the minimum time between an ACT command and a column access (read/write) within the same bank.

tRP

tRP specifies the minimum time between a PRE command and any other command to the same bank.

tRAS/tRC

tRAS specifies the minimum time between an ACT and PRE command to the same bank. tRC specifies the minimum time between two ACT commands to the same bank. Typically, DDR3 SDRAM is limited by the time between two ACT commands, and not the time between an ACT and PRE command; however, since the minimum time between two ACT commands (tRC) will be limited by the minimum time between ACT and PRE (tRAS) plus the minimum time between PRE and any other command (tRP), the minimum effective tRC will be $tRAS + tRP$.

Due to this redundancy, Intel processors have excluded the tRC timing and simply use tRAS (in combination with tRP) to control the effective tRC. As a result of this, you will be able to trade tRP for tRAS when tRAS is pushed to the limit. Since the tRP timing has a greater impact on performance than tRAS, make sure to tune tRP before tRAS to avoid such a scenario.

tRTP

tRTP specifies the minimum time between a Read and PRE command to the same bank. Since tRTP is typically lower than tCAS, this means that a row may be closed before the data read from it has "left" the DRAM.

tWR

tWR specifies the minimum time between the end of data input (end of a write burst) and a Precharge command. Since read and write bursts last 4 clock cycles, and write bursts start CWL clock cycles after a Write command, the effective minimum time between a Write and PRE command is $tWR + CWL + 4$.

tWTR

tWTR is the minimum time between the end of a write burst (end of data being sent to the DRAM) and a Read command.

On Intel, tWTR itself is not an actual timing, and the delay is instead controlled by the tWRRD timings, where tWRRD_sg corresponds to tWTR_L and tWRRD_dg to tWTR_S. They are related in the following manner:

$$tWRRD = tWTR + CWL + 4$$

Note that the reported tWTR values in programs such as Asrock Timing Configurator on Intel motherboards are 2 lower than the actual effective values. Thus, you may see reported timings of 0 or other odd values, especially for tWTR_S on Micron and Hynix ICs.

tREFI

tREFI specifies the maximum average time between Refresh commands. For performance reasons, up to 8 Refresh commands may be postponed. Likewise, up to 8 Refresh commands may be "pulled in" (issued in advance), allowing for a longer period without the need for refreshing later. The maximum time between two consecutive Refresh commands may not exceed $9 \times tREFI$.

tRFC

tRFC specifies the minimum time that must elapse between a Refresh command and any other command. As evident by this, refreshing "wastes" time that could be used for other operations; thus, lower tRFC and higher tREFI lead to better performance. The portion of time spent waiting for refreshing will be $tRFC/tREFI$.

Micron revisions and design IDs

Micron is known to use design IDs internally for their IC revisions and prototypes. These are also the revisions/IDs that get used for SpecTek, as well as what Corsair and sometimes G.skill use for their IC identification codes (Z11C being 3.32 for C, for example).

Revision	Design ID (DID)	Node
4 Gbit revision A	Z80A	30 nm
4 Gbit revision B	Z90B	25 nm
4 Gbit revision F	Z10B	1x
4 Gbit revision G?	Z20A	1y
8 Gbit revision A	Z91B	25 nm
8 Gbit revision B	Z01A	20 nm
8 Gbit revision D	Z01B	20 nm
8 Gbit revision E	Z11B	1x
8 Gbit revision G	Z01C	20 nm
8 Gbit revision H	Z11C	1x
8 Gbit revision J	Z21C	1y
8 Gbit revision R	Z41C	1a
16 Gbit revision B	Z22A	1y
16 Gbit revision E	Z32D	1z
16 Gbit revision F	Z42B	1a

Sources: Various Micron documents, PCNs and roadmaps, as well as TechInsights. May add full source list in the future.

Meme timing rules

I will briefly cover some made-up timing rules with the goal of stopping the spread of misinformation by explaining the likely origin of these timing rules and why they are incorrect.

tWR = 2x tRTP

This one originates from the fact that these timings are by JEDEC spec (in other words, auto values) are related in the stated manner. However, overclocking is by definition exceeding the specification, and values stated in the specification are thus no longer relevant. These two

timings are not related in any way. They simply happen to in some cases run minimum values that abide by the above relationship.

CL = tRCD = tRP

This rule is largely a parody of the above timing rule, since it also applies according to the specification. However, as observed in practice, no one abides by this rule, further proving the point of spec values being irrelevant when overclocking.

tRAS = tRCD + tRTP

This timing rule is likely based on incorrect assumptions made by people with little or no experience of reading well-defined specs or datasheets. JESD79-4 includes a figure which may to some readers hint that tRAS lower than tRCD + tRTP would serve no purpose. However, this is an extrapolation of a statement which has not been made about a situation which is not exclusive. By spec, tRAS is always higher than tRCD + tRTP anyways, so there is no need to include specific statements about operation at low tRAS in the spec. In practice, a memory controller might want to close a row shortly after opening it, likely as a consequence of incorrect predictive row opening, in which case tRAS lowering tRAS below tRCD + tRTP would yield benefits. However, diminishing returns would be expected from tRAS lower than tRCD + tRTP.

Proof of tRAS lower than tRCD + tRTP yielding actual effects is confirmed by both slight performance benefits in some workloads, and observed instability at tRAS < tRCD + tRTP. If it did not have an effect, instability would not be observed and performance would not improve.

tRFC = n * tRC

Often this will be in the form of tRFC = 8x tRC or tRFC = 7x tRC. I am unsure of what the origin for this one is, other than people trying too hard to observe patterns in things which are not related. There is no evidence for any such relation and there is to my knowledge not even any proper information that could reasonably be interpreted as hinting at such a relation.

tRRD_L = tRDRD_sg

This one is likely based on the fact that the minimum configurable value (and sometimes overclocked value) for these two is the same. Again, these timings are completely unrelated and any implication that they are is likely a result of too much effort put into finding patterns in unrelated parameters.

tREFI = n * tRFC

This one is straight up wrong. Just because the timings control latencies related to the same operation does not mean the timings themselves are related.

tREFI = tREFI₉ / n

This one seems to be based on a misunderstanding of how DDR4 refreshing actually works; an integer number of refreshes do not need to fit into the maximum window between any two consecutive refreshes. No such mention is made in JESD79-4. The only remotely similar

observation which holds true is that tREFI higher than effective tREFI_{x9} does not yield any benefits (per definition).

$$tWR = 6 * n$$

I have no idea what the origin of this one is. It doesn't even make any sense.

$$tWTR_L = 2x\ tRRD_L\ \text{and}\ tWTR_L = 4x\ tWTR_S$$

Again, no idea what the origin or rationale of these are.

$$tRDWR = CL - CWL + 8$$

This is a rule-of-thumb, not a strict rule; some people seem to have misunderstood this. While this formula gives approximate tRDWR values to try, they don't need to strictly abide by it. CL - CWL + 9 is also common, especially for higher frequencies.

Changelog

V1.2.0

Changes

- Added recommendations for tuning tRCD(WR)
- Added more detailed info for Micron and Samsung 16 Gbit ICs
- Added 8 Gbit Samsung E-die
- Fixed a few typos (1766 instead of 1866 MHz and tWTR_L instead of S)
- Corrected max tREFI for Intel 11th gen
- Corrected non-recommended MemTest86 test from 11 to 13
- A few clarifications added
- Some minor word choice changes

V1.1.0

Changes

- Stability testing section greatly expanded in depth
- Advanced tweaks section added, now with Ryzen info
- Clarified that people should actually stability test every now and then
- tWTR timing explanation added
- tWRRD section cleaned up, with some info moved to the tWTR timing explanation
- More/better info on B-die voltage scaling
- Various minor changes

V1.0.0 (first full release)

Changes

- Ryzen frequency information
- More IC information
- 6th–10th gen tweaks
- Micron design IDs and nodes
- Various minor changes

V0.3.0

Changes

- Tips and troubleshooting
- Termination resistances
- Meme timing rules and why they don't apply
- Slightly more IC information

Planned additions:

- Rank and IMC load section with more details
- Possibly Ryzen info (volunteers are welcome)
- More IC info will be added as I feel like it. Will not include as planned anymore.

V0.2.0

Changes

- Various minor changes
- IC identification guide added
- Timing explanations added

Planned additions:

- See previous version, minus the ones added
- FAQ/troubleshooting

V0.1.0

Changes

- First public release

Planned additions:

- IC identification guide
- Possibly Ryzen info
- More IC info
- More rank and IMC load info
- Termination resistances
- Potentially IC rankings if there is demand (though that is quite subjective)
- Timing explanations if there is demand (could just copy it from my DDR3 guide)
- List of made up meme rules (For the memez! Good idea or nah?)